

Project Clara:

AI Balance Nutrition Meal Planner

Brainstorming - [Miro Dashboard](#)

1. Summary

Clara AI is an intelligent, personalized meal planning engine built to automate the creation of accurate diet plans using the existing BN recipes . By integrating AI with a robust (**Hybrid Retrieval-Augmented Generation (RAG)** architecture) - **Recipe recommendation system** , Clara AI dynamically translates complex user profiles including metabolic data, strict dietary restrictions, and User preferences into precise, actionable nutrition Meal plans.

The Core Value Proposition: Moving away from static templates and randomized recipe selection toward a context-aware, user preference validated, and real-time automated nutrition engine.

2. Project Goals & Success Metrics

- **Automated Hyper-Personalization:** Real-time adaptation of meal plans based on dynamic user metrics (BMI, BMR, personal preferences, and lifestyle factors).
- **High-Fidelity Recipe Retrieval:** Precision-targeted sourcing from the Balance Nutrition (BN) Recipe Pool using combined semantic and keyword matching.
- **Scalability & Cost Efficiency:** Minimization of LLM token overhead and API latency via aggressive data caching and optimized context windows.

3. Technical Architecture & Tech Stack

The system is split into a highly responsive frontend, an orchestrated backend, and an isolated, containerized database layer.

Component	Technology	Role & Key Responsibility
Frontend	TypeScript / React	Handles complex user onboarding (OnboardingForm), toggle states (e.g., Jain diet rules), and interactive Food Recall UIs.

Backend	Node.js (Clean <code>src/</code> Architecture)	Manages API routing, data validation, AI integration pipelines, and prompt orchestration.
Database	Qdrant (Vector Database)	Hosted locally; manages high-dimensional recipe vector embeddings for dense semantic search.
Infrastructure	Docker & Docker Compose	Multi-container isolation ensuring a fully reproducible local and production deployment environment.

4. Deep Dive: Phase 2 Hybrid RAG System

To ensure recipe selections are both contextually appropriate and mathematically accurate, Clara AI utilizes a **Three-Tiered Retrieval Pipeline**.



Retrieval Breakdown

1. **Dense Vector Search (Qdrant):** Parses user context to find conceptually similar items (e.g., linking a "keto morning meal" request directly to a "healthy egg scramble").
2. **Sparse Search (BM25):** A pure JavaScript implementation handling strict keyword logic. This acts as a hard filter to guarantee forbidden ingredients are entirely omitted.
3. **Cross-Encoder Reranking (Gemini):** Evaluates combined candidates from both searches, scoring and ordering the top results based on nuanced clinical and contextual alignment before passing them to the final prompt.

Token & Performance Optimization

- **Vault Restrictions:** Tight caps on the number of candidates forwarded to the reranker stage.
- **Payload Truncation:** Drops non-essential metadata, passing only critical ingredients and core nutritional metrics to the LLM context window to reduce latency and API costs.

5. Diet Creation Parameters & Variables

The orchestration engine ingests, calculates, and maps a multi-layered matrix of user data points before passing instructions to the LLM. This ensures the generated diet is mathematically sound and clinically safe.

A. User Demographics & Anthropometrics

- **Core Metrics:** Age, Gender, Height, and Weight are captured to compute precise basal metabolic rates (BMR).
- **Clinical Categorization:** Calculated BMI and BMI Class provide the necessary clinical context regarding the user's weight category (e.g., Underweight, Normal, Overweight, Obese Class I/II/III).

B. Dietary & Clinical Preferences

- **Diet Type Pre-Filtering:** Classifications such as *Vegetarian*, *Non-Vegetarian*, or *Jain* trigger strict database pre-filtering. This rule applies at the database layer before the LLM evaluates any recipes.
- **Objective-Driven Guidance:** Goals like *Weight Loss*, *Muscle Gain*, or *Maintenance* trigger targeted clinical strategies (e.g., high-protein/high-fiber prompts for loss vs. calorie-dense configurations for gain).
- **Guardrails & Taste Mapping:** * *Allergies & Aversions:* Blacklisted ingredients are strictly enforced and scrubbed from retrieval pipelines to guarantee safety.
 - *Cuisine & Ethnicity:* Maps regional taste preferences.
 - *Food Recall UI:* Integrates specific historical meal choices specified by the user across Breakfast, Lunch, Snack, and Dinner slots to preserve user habits.
 - *Health Conditions:* Maps chronic or acute clinical flags directly to targeted prompt constraints.

C. Calculated Nutritional Targets

The engine divides daily nutritional targets into structured meal-specific budgets:

- **Daily Caloric Ceiling:** Total daily energy expenditure targets calculated using the **Mifflin-St Jeor equation**.
- **Dynamic Macro Splits:** Real-time calculation of exact gram targets for Protein, Carbohydrates, and Fats based on the user's overarching health goal.
- **Per-Meal Allocations:** No of meals consumed + Daily targets are distributed across the day using percentage budgets:

Meal Slot	Caloric & Macro Allocation (%)
Breakfast	25%
Lunch	30%
Snack	15%
Dinner	30%

D. "Mentor" Rules (Extra)

- **Weight Threshold Rule:** * If user weight is ≥ 100 kg: The LLM is forced to structure Lunch and Dinner to include **Salad AND Sabzi AND Roti**.
 - If user weight is < 100 kg: The rule relaxes to **(Salad OR Sabzi) AND Roti**.
- **Multi-Option Mandates:** To prevent plan fatigue, the system enforces structured variety within a single day's plan:
 - *Breakfast:* Exactly 3 distinct choices (Salt-free, Sugar-free, Regular).
 - *Lunch:* Exactly 2 distinct choices (Rice-based vs. Roti-based).

7. LLM Payload (Data Pipeline Output)

To minimize token consumption, maximize context-window efficiency, and enforce absolute generation accuracy, Clara AI compiles all variables into a single, highly structured system prompt.

The LLM does not independently browse or guess recipes. It acts as a compiler for the following structured payload:

1. Formatted Client Profile
(Demographics, Preferences, Allergies, Food Recall)
2. Strict Nutritional Budget |
(Exact calorie/macro gram targets per meal slot)
3. Injected Clinical Guidance |
(Goal-specific advice, Weight-based Mentor Rules)
4. Pre-Filtered "Recipe Vault" |
(~35 optimized, pipe-separated RAG candidates)
5. Strict Instruction Set |
(Zero-hallucination rules, serving size multipliers)|

The Recipe Vault Format

The Hybrid RAG system compresses the top ~35 retrieved and reranked recipe candidates into a compact, pipe-separated string structure. This prevents token bloat from raw JSON formats:

slug | recipe name | calories | protein | carbs | fat | category | RAG Relevance Score

Generation Execution Constraints

- **Zero-Hallucination Guardrail:** The LLM is strictly forbidden from inventing recipes. It can *only* select items explicitly provided within the injected Recipe Vault string.
- **Component Combination:** For composite meals (like Lunch and Dinner), the LLM must mathematically combine 2 to 3 recipes from the vault to hit the meal slot's exact macro targets.
- **Serving Size Hydration:** If a high-relevance recipe is nutritionally insufficient on its own, the LLM must calculate and output precise serving size multipliers (e.g., **2.5 servings**) to scale the recipe accurately to the meal budget.

8. API Architecture & Endpoint Specifications

Clara AI exposes a structured REST API layer divided into functional domains: Core Planning, Smart Shopping Intelligence, and System Utilities. The core generation pipeline utilizes real-time streaming to optimize perceived latency on the frontend.

A. Core Diet & Meal Planning Engines

These endpoints handle the orchestration of the Hybrid RAG engine, recipe discovery, and real-time plan modification.

Endpoint	Method	Description	Key Mechanism
/api/generate-plan	POST	Orchestrates full AI meal plan generation using the user profile. Streams a 3-day meal plan back to the frontend.	Server-Sent Events (SSE) for real-time generation streaming.
/api/swap-options	POST	Provides context-aware alternative meal options if a user rejects a generated recipe.	Regenerates options within the exact same meal slot macro/calorie budget.
/api/recipes	GET	Fetches the full index of validated recipes from the backend database/store.	Sourced directly from the Balance Nutrition (BN) Recipe Pool.

B. Smart Shopping Intelligence

An auxiliary layer designed to improve user adherence by simplifying grocery shopping and managing nutritional gaps between major meals.

- **POST /api/shop-alternatives**
 - **Purpose:** Evaluates items within a user's grocery basket.
 - **Action:** Employs AI to identify non-compliant ingredients or lower-tier choices and suggests healthier, profile-approved alternatives.

- **POST /api/quick-fillers**
 - **Purpose:** Generates a dynamic list of rapid-assembly or grab-and-go snack items.
 - **Action:** Constrains suggestions strictly to the user's remaining daily calorie/macro allowances and medical filters.

C. Utility & System Orchestration

Endpoints dedicated to microservice health telemetry, accessibility features, and downstream integration mapping.

- **GET /api/health**
 - Standardized shallow health check endpoint used to verify that the Node.js backend application instance is live and accepting traffic.
- **POST /api/generate-speech**
 - A Text-to-Speech (TTS) engine utility that converts text-based diet guidelines, mentor notes, or recipe instructions into audio formats for enhanced frontend accessibility.
- **GET /api/upstream-status**
 - Performs deep health checks against critical external dependencies. It monitors the connectivity, response latency, and authorization state of the **BN Client API** and the external **Recipe API**.